Individual Assignment: Mesh Graph Networks for Flow Field Prediction

Objective

Implement a Mesh Graph Network (MeshGraphNet) to predict mean pressure and velocity fields for flow past centered obstacles. You will work incrementally, building from data preprocessing to a complete graph neural network architecture. The design can follow closely the lecture material on the topic.

Overview

Dataset: You are provided with flow simulations past centered obstacles in .vtu format. Each simulation contains mesh geometry (nodes and connectivity) and mean flow fields: velocity components (u, v) and pressure p.

Signed Distance Function: The signed distance function (SDF) measures the distance from each mesh node to the nearest boundary, with sign indicating interior vs. exterior regions. Nodes are classified into types (boundaries, inlet, outlet, interior). Boundaries include the walls and obstacle.

Graph Neural Networks for Mesh-Based Physics: MeshGraphNet uses message passing on mesh connectivity to learn physical dynamics. The architecture consists of:

- **Encoder**: MLP that embeds node features into latent space
- **Processor**: Message passing layers that propagate information across the mesh
- **Decoder**: MLP that maps latent representations to predicted fields

Q1. Data Preprocessing and Signed Distance Function

Extract boundary points from the VTU meshes, classify node types, and compute the signed distance function (SDF). Pyvista library is the most appropriate for handling simulation data of this format. Example of loading a mesh of this type and its nodes on the boundary:

```
import pyvista as pv
mesh = pv.read(vtu_path)
boundary = mesh.extract_feature_edges().points
```

Tasks:

- Extract boundary points from all meshes in your dataset
- Classify nodes into types based on domain geometrical characteristics: boundary = 0, interior = 1, inlet = 2, outlet=3
- Calculate SDF for each mesh node using 2D coordinates (x, y). You may utilize `cKDTree(boundary_points)` to compute distances efficiently
- Visualize: (1) SDF values across the mesh, (2) node type classification

Q2. Model Implementation

Implement the MeshGraphNet architecture incrementally. Utilize PyTorch Geometric.

Construction of Graph Data object:

- Convert mesh to `torch_geometric.data.Data` format
- Node features: coordinates, SDF, node type
- Edges from mesh connectivity, with node distance and relative position as features
- Targets: velocity (u, v) and pressure p
- Implement collate function for batching (how does this data format handle batching?)

Model Components:

Encoder: MLP with at least 2 hidden layers to embed input features to latent dimension.

```
class Encoder(nn.Module):
    def __init__(self, input_dim, latent_dim, hidden_dim):
        # TODO: Implement MLP encoder
```

Processor: Custom message passing layer, with dynamic depth. Include option to choose between Graph Convolution Layer (`GCNConv`) and your custom message passing implementation.

```
class Processor(nn.Module):
    def __init__(self, latent_dim):
        # TODO: Series of Graph Neural Network Layers. You may
            implement the MessagePassingLayer module in a standalone
            class
```

Decoder: MLP to predict 3 output values per node (u, v, p).

```
class Decoder(nn.Module):
    def __init__(self, latent_dim, output_dim, hidden_dim):
        # TODO: Implement MLP decoder
```

Q3. Training and Evaluation

Train three model configurations and compare performance using Relative Root Mean Square Error (RRMSE):

$$\text{RRMSE} = \frac{\sqrt{\frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2}}{\sqrt{\frac{1}{N}\sum_{i=1}^{N} y_i^2}} \tag{1}$$

Configuration 1 – Baseline: No encoder, feed raw features directly to processor with custom message passing and use decoder to output field predictions.

Configuration 2 – GCN Processor: Use encoder, processor and decoder architecture from Q2, but replace custom processor with `GCNConv` from PyTorch Geometric.

Configuration 3 – Complete MeshGraphNet: Full architecture with encoder, custom processor, and decoder.

For each configuration you need to experiment with number of message passing layers (5, 10, 15 or more) and latent dimensions (16, 32, 64, 128). Make sure to split your dataset to train and test data and evaluate on the test set. For the training you may use a Mean Square Error (MSE) loss function or any other that you see fit. For each configuration report:

- RRMSE for u-velocity, v-velocity, and pressure
- Training scheme: hyperparameters, epochs and callbacks
- Visualization: For 2 test samples, plot ground truth and predicted fields for u, v, and p

Compare the three configurations. Which architecture performed best and how do you explain the result?

Q4. Bonus Question

Implement edge subsampling to reduce graph connectivity during training. Randomly subsample edges from the mesh graph at varying ratios (e.g., 0.05, 0.1, 0.25) while keeping all nodes. Train your best model configuration from Q3 on these subsampled graphs using the full training set, then evaluate on the original fully-connected test meshes. Report RRMSE metrics for each subsampling ratio and discuss how sparse connectivity affects prediction performance.